

[КАК СТАТЬ АВТОРОМ](#)[Курсы для тестировщиков](#)[Какой вы ML-специалист: ...](#)**bicikar**

27 мая 2022 в 11:31

Подгон под MNIST-овский датасет



7 мин



9.9K

Python*, TensorFlow*, Машинное обучение*

[Из песочницы](#)[Перевод](#)

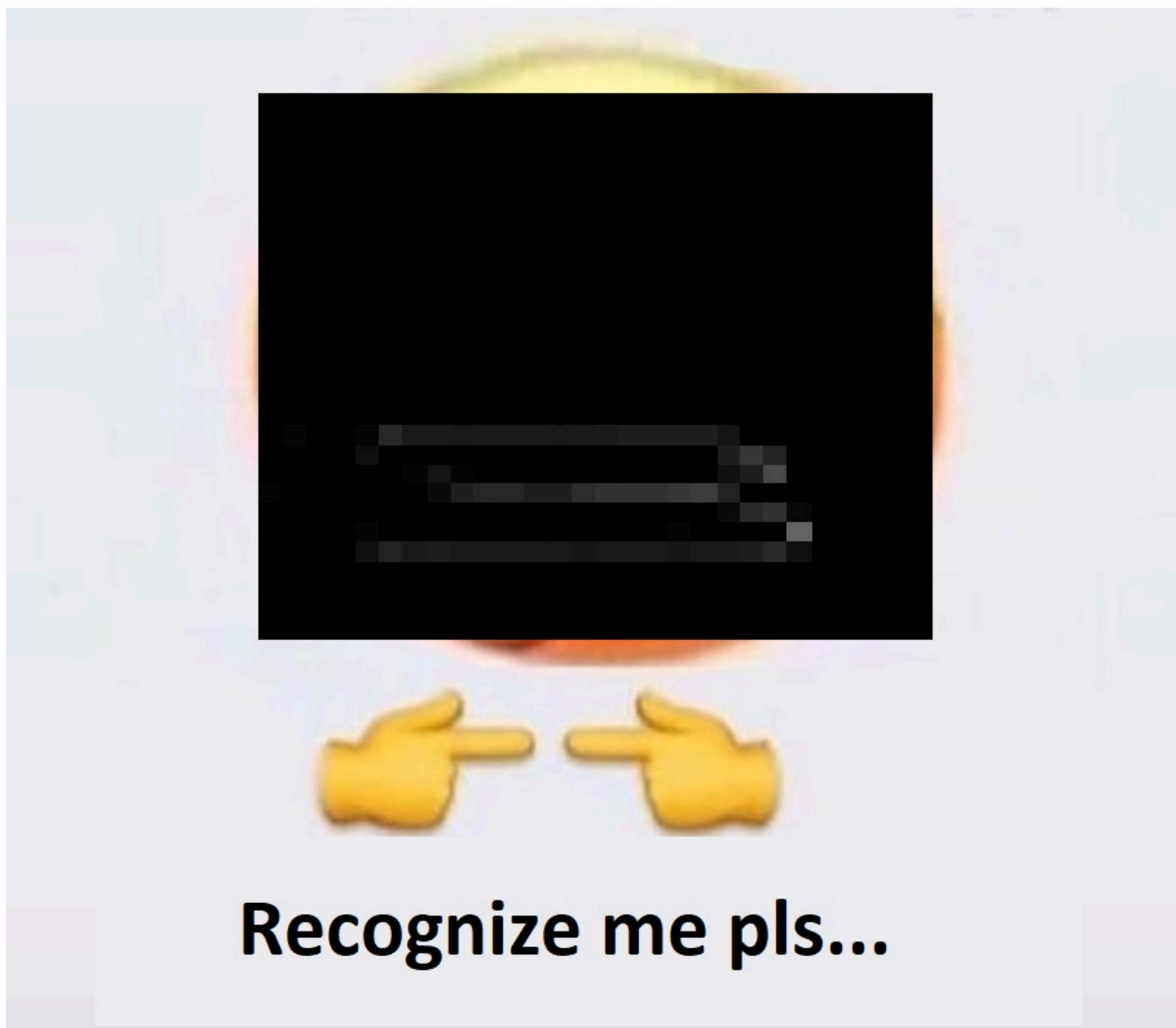
Автор оригинала: Ole Krömlger

В интернете можно найти 1000 и 1 статью по тренингу мнистовского датасета для распознавания рукописных чисел. Однако когда дело доходит до практики и начинаешь распознавать собственные картинки, то модель справляется плохо или не справляется вовсе. Конечно же мы можем перевести картинку в оттенки серого, насильно поменять размер под мнистовский на 28x28 пикселей, и тогда наша сеть будет работать с подобными картинками:

РЕКЛАМА

**Облачный вызов**

Соревнование разработчиков



Естественно, основная проблема состоит в том, что произвольная картинка сильно отличается от базы картинок MNIST. Исходные MNIST-овские цифры помещаются в квадратную картинку 20x20 пикселей. Затем вычисляется центр масс изображения и оно располагается на поле размера 28x28 пикселей таким образом, чтобы центр масс совпадал с центром поля. Именно к такому виду мы и должны подгонять наши данные.

Используем любую реализацию модельки для распознавания MNIST-овских чисел. Например:

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```



Облачный вызов

Соревнование разработчиков

```
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
```

```
from tensorflow.keras.optimizers import SGD

# load train and test dataset
def load_dataset():
    # load dataset
    (trainX, trainY), (testX, testY) = mnist.load_data()
    # reshape dataset to have a single channel
    trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
    testX = testX.reshape((testX.shape[0], 28, 28, 1))
    # one hot encode target values
    trainY = to_categorical(trainY)
    testY = to_categorical(testY)
    return trainX, trainY, testX, testY

# scale pixels
def prep_pixels(train, test):
    # convert from integers to floats
    train_norm = train.astype('float32')
    test_norm = test.astype('float32')
    # normalize to range 0-1
    train_norm = train_norm / 255.0
    test_norm = test_norm / 255.0
    # return normalized images
    return train_norm, test_norm

# define cnn model
def define_model():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
    model.add(MaxPooling2D((2, 2)))
    model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
    model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
    model.add(MaxPooling2D((2, 2)))
    model.add(Flatten())
    model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
    model.add(Dense(10, activation='softmax'))
    # compile model
    opt = SGD(learning_rate=0.01, momentum=0.9)
    model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```



Облачный вызов

Соревнование разработчиков

```
trainX, trainY, testX, testY = load_dataset()
# prepare pixel data
```

```
trainX, testX = prep_pixels(trainX, testX)
# define model
model = define_model()
# fit model
model.fit(trainX, trainY, epochs=10, batch_size=32, verbose=1)
# save model
model.save('digit_model.h5')
_, acc = model.evaluate(testX, testY, verbose=0)
print('> %.3f' % (acc * 100.0))

# entry point, run the test harness
run_test_harness()

>>> 99.040
```

Получили довольно неплохой ассигасу. Теперь возьмем наши личные картинки и посмотрим, что нам предоставит сеть. Самый стандартный способ препроцессинга: заскейлить до 28 пикселей, инвертировать цвет:

```
import cv2
import numpy as np

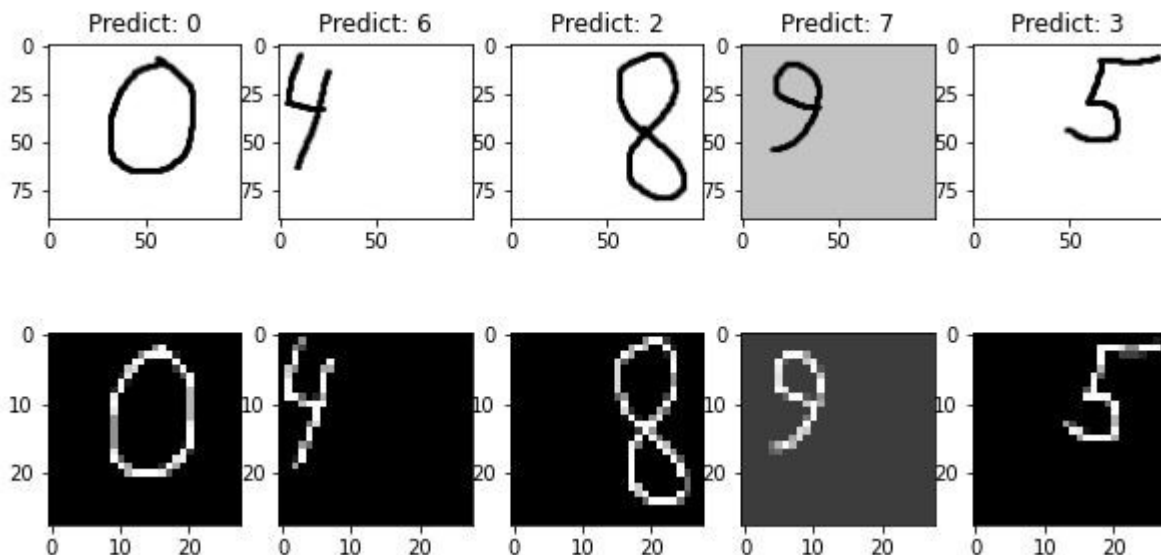
def rec_digit(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gray = 255 - img

    gray = cv2.resize(gray, (28, 28))
    cv2.imwrite('gray'+ img_path, gray)
    img = gray / 255.0
    img = np.array(img).reshape(-1, 28, 28, 1)
    out = str(np.argmax(model.predict(img)))
    return out
```



Облачный вызов

Соревнование разработчиков



Ноль распознанся нормально, потому что находится по центру и расположен в целом довольно удачно. С остальными числами плохо. Получается, точность на 5 тестовых картинках всего 20 процентов.

Еще раз сформулируем главный тезис, как выглядит **датасет**: The original black and white (bilevel) images from NIST were size normalized to fit in a 20x20 pixel box while preserving their aspect ratio. The resulting images contain grey levels as a result of the anti-aliasing technique used by the normalization algorithm. the images were centered in a 28x28 image by computing the center of mass of the pixels, and translating the image so as to position this point at the center of the 28x28 field.

Преобразуем все картинки к такому формату. Также отметим, что если фон не совсем белый, то мы получим что-то, сильно отличающееся от мнистовского датасета, белая цифра на черном фоне, как в примере с девяткой. Поэтому добавим пороговую обработку после считывания изображения:

```
def rec_digit(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gray = 255-img
    # применяем пороговую обработку
    (thresh, gray) = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

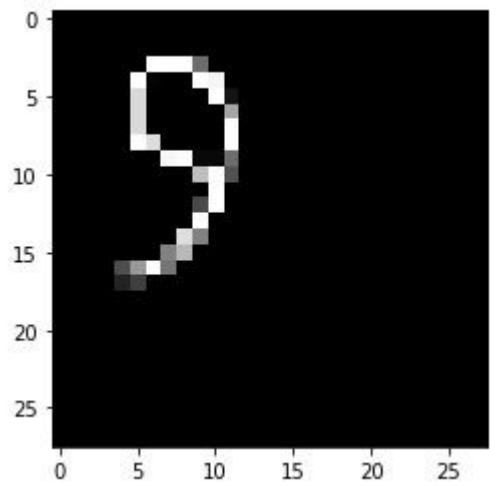
    gray = cv2.resize(gray, (28, 28))
    cv2.imwrite('gray'+ img_path, gray)
    img = gray / 255.0
```



Облачный вызов

Соревнование разработчиков

```
return out
```



После применения пороговой обработки

Теперь мы хотим поместить картинку в бокс 20x20 пикселей. Можно делать это несколькими способами. Один из вариантов - найти контур, ограничивающий цифру, взять его в качестве основного изображения и сделать **resize** до нужных размеров. Пример, как это можно делать. В том числе может пригодиться, если необходимо распознавать числа из более чем одной цифры.

Мы же будем делать немного проще и, с другой стороны, надежнее. А именно, сперва удалим все строки и столбцы, в которых пиксели только черные. Таким образом мы получим картинку, в точности являющейся прямоугольной оболочкой нашей цифры.

```
def rec_digit(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gray = 255-img
    # применяем пороговую обработку
    (thresh, gray) = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

    # удаляем нулевые строки и столбцы
    while np.sum(gray[0]) == 0:
        gray = gray[1:]
    while np.sum(gray[:,0]) == 0:
        gray = np.delete(gray,0,1)
    while np.sum(gray[-1]) == 0:
        gray = gray[:-1]
    while np.sum(gray[:, -1]) == 0:
        gray = np.delete(gray, -1, 1)
    rows, cols = gray.shape
```

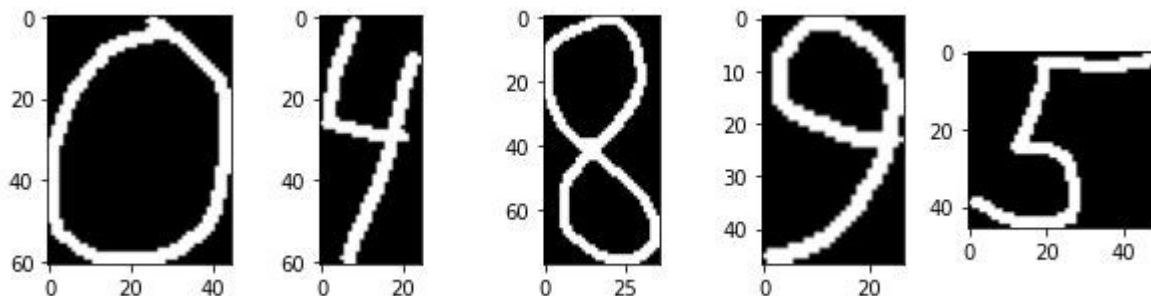


Облачный вызов

Соревнование разработчиков

```
img = gray / 255.0
img = np.array(img).reshape(-1, 28, 28, 1)
```

```
out = str(np.argmax(model.predict(img)))
return out
```



В целом, в точности и получаются bounding boxes.

Далее хотим поменять размер картинок таким образом, чтобы они помещались в квадрат размера 20x20. Добавим фактор, чтобы наибольшая сторона была длиной в 20 пикселей:

```
def rec_digit(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gray = 255-img
    # применяем пороговую обработку
    (thresh, gray) = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

    # удаляем нулевые строки и столбцы
    while np.sum(gray[0]) == 0:
        gray = gray[1:]
    while np.sum(gray[:,0]) == 0:
        gray = np.delete(gray,0,1)
    while np.sum(gray[-1]) == 0:
        gray = gray[:-1]
    while np.sum(gray[:, -1]) == 0:
        gray = np.delete(gray, -1, 1)
    rows, cols = gray.shape

    # изменяем размер, чтобы помещалось в box 20x20 пикселей
    if rows > cols:
        factor = 20.0/rows
        rows = 20
        cols = int(round(cols*factor))
```



Облачный вызов

Соревнование разработчиков

```
factor = 20.0/cols
cols = 20
rows = int(round(rows*factor))
```

```

gray = cv2.resize(gray, (cols, rows))

cv2.imwrite('gray'+ img_path, gray)
gray = cv2.resize(gray, (28, 28))
img = gray / 255.0
img = np.array(img).reshape(-1, 28, 28, 1)
out = str(np.argmax(model.predict(img)))
return out

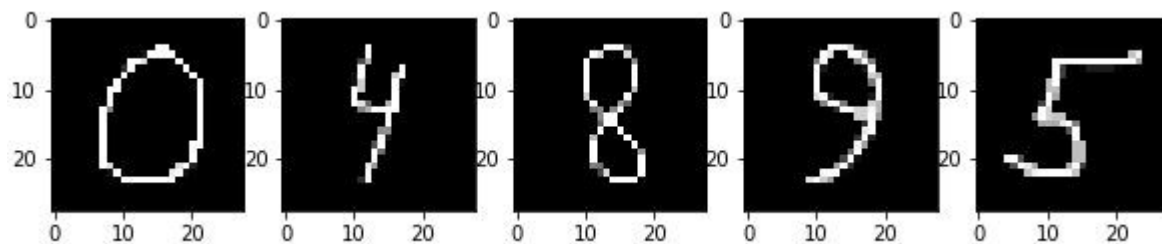
```

Теперь расширяем картинку до 28x28 пикселей, добавляя черные ряды и столбцы по краям, используя функцию `np.lib.pad`, которая добавляет нули по краям. И сразу удалим строчку `gray = cv2.resize(gray, (28, 28))`. После факторизации добавляем:

```

colsPadding = (int(math.ceil((28-cols)/2.0)),int(math.floor((28-cols)/2.0)))
rowsPadding = (int(math.ceil((28-rows)/2.0)),int(math.floor((28-rows)/2.0)))
gray = np.lib.pad(gray,(rowsPadding,colsPadding),'constant')

```



Добавили границы до размера 28x28

В целом, картинки уже расположены довольно хорошо. Тем не менее, следующий шаг - подвинуть внутреннюю коробку таким образом, чтобы ее центр масс совпадал с центром всей картинки. Заведем две вспомогательные функции. Первая вычисляет центр масс и направление сдвига:

```

from scipy.ndimage.measurements import center_of_mass
def getBestShift(img):
    cy,cx = center_of_mass(img)

```



Облачный вызов

Соревнование разработчиков

```

return shiftx,shifty

```

И собственно функция, которая сдвигает картинку в нужном направлении. Подробнее о `warpAffine`. В нашем случае следующая матрица трансформации:

$$M = \begin{pmatrix} 1 & 0 & s_x \\ 0 & 1 & s_y \end{pmatrix}$$

```
def shift(img,sx,sy):
    rows,cols = img.shape
    M = np.float32([[1,0,sx],[0,1,sy]])
    shifted = cv2.warpAffine(img,M,(cols,rows))
    return shifted
```

Добавляем еще пару строчек со сдвигом относительно центра масс:

```
shiftx,shifty = getBestShift(gray)
shifted = shift(gray,shiftx,shifty)
gray = shifted
```

И по итогу получаем полноценный подгон под мнистовский датасет:

```
from scipy.ndimage.measurements import center_of_mass
import math
import cv2
import numpy as np

def getBestShift(img):
    cy,cx = center_of_mass(img)

    rows,cols = img.shape
    shiftx = np.round(cols/2.0-cx).astype(int)
    shifty = np.round(rows/2.0-cy).astype(int)

    return shiftx,shifty
```



Облачный вызов

Соревнование разработчиков

```
M = np.float32([[1,0,sx],[0,1,sy]])
shifted = cv2.warpAffine(img,M,(cols,rows))
return shifted
```

```
def rec_digit(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    gray = 255-img
    # применяем пороговую обработку
    (thresh, gray) = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

    # удаляем нулевые строки и столбцы
    while np.sum(gray[0]) == 0:
        gray = gray[1:]
    while np.sum(gray[:,0]) == 0:
        gray = np.delete(gray,0,1)
    while np.sum(gray[-1]) == 0:
        gray = gray[:-1]
    while np.sum(gray[:, -1]) == 0:
        gray = np.delete(gray, -1, 1)
    rows, cols = gray.shape

    # изменяем размер, чтобы помещалось в box 20x20 пикселей
    if rows > cols:
        factor = 20.0/rows
        rows = 20
        cols = int(round(cols*factor))
        gray = cv2.resize(gray, (cols,rows))
    else:
        factor = 20.0/cols
        cols = 20
        rows = int(round(rows*factor))
        gray = cv2.resize(gray, (cols, rows))

    # расширяем до размера 28x28
    colsPadding = (int(math.ceil((28-cols)/2.0)),int(math.floor((28-cols)/2.0)))
    rowsPadding = (int(math.ceil((28-rows)/2.0)),int(math.floor((28-rows)/2.0)))
    gray = np.lib.pad(gray,(rowsPadding,colsPadding),'constant')

    # сдвигаем центр масс
    shiftx,shifty = getBestShift(gray)
    shifted = shift(gray,shiftx,shifty)
    gray = shifted

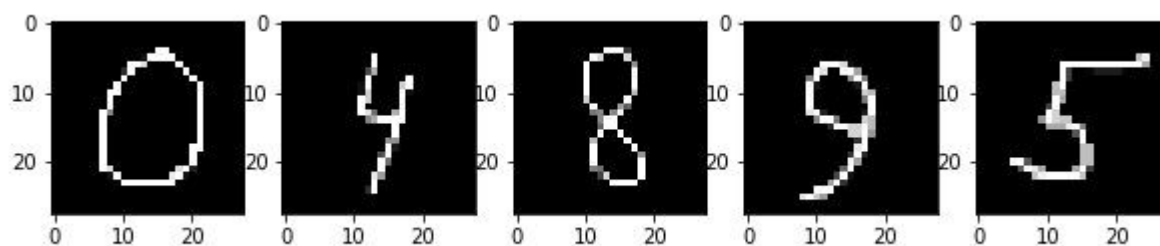
    cv2.imwrite('gray'+ img_path, gray)
```

**Облачный вызов**

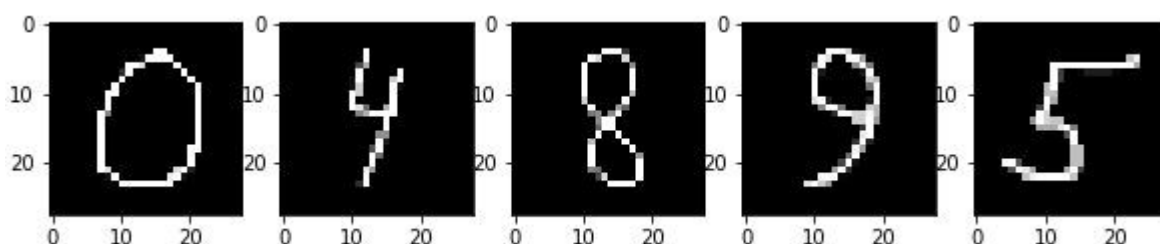
Соревнование разработчиков

```
return out
```

Вообще можно было бы задаться вопросом, действительно ли сдвиг относительно центра масс имеет вообще хоть какой-то толк, особенно если мы работаем с картинкой 20x20 пикселей? Разница будет, пусть и небольшая. Тем не менее, мы подогнали произвольную картинку под MNIST-овский датасет.

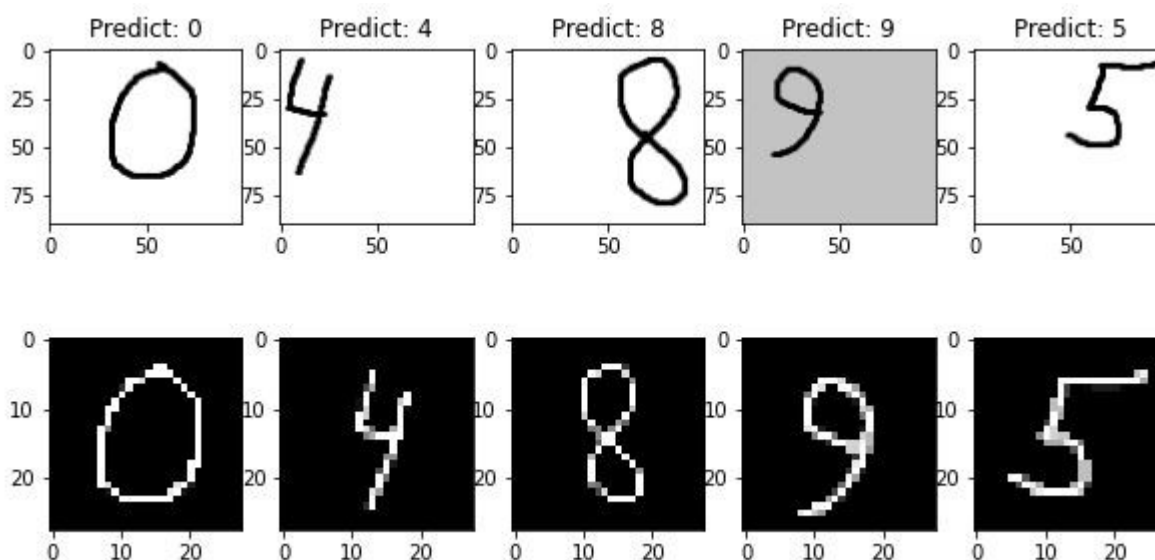


Изображения со сдвигом центра масс



Изображения до сдвига центра масс, после добавления границ до размера 28x28

Как итог, моделька выше с использованием построенного препроцессинга изображений дает следующий результат:



Облачный вызов

Соревнование разработчиков

Теги: mnist, рукописные числа, neural networks, handwritten digits

Хабы: Python, TensorFlow, Машинное обучение

 +8 34 0

Редакторский дайджест



Присылаем лучшие статьи раз в месяц

Оставляя свою почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



3

Карма

0

Рейтинг

@bicikar

Пользователь

[Подписаться](#)

Облачный вызов

Соревнование разработчиков

Комментарии



Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

**vvvphoenix**

19 часов назад

OpenCV. Начало**Простой**

13 мин



9.6K

[Мнение](#)[Космотекст](#)**+140**

52



9

**k0mar0v**

22 часа назад

Интернет отключили, а ты в чате: что такое VarAC и при чем здесь коротковолновое радио

7 мин



11K

**+55**

72



41

**empenoso**

10 часов назад

Экологичное расставание с Алисой: строим полностью локальный и приватный голосовой ассистент**Простой**

8 мин



16K

**Облачный вызов**

Соревнование разработчиков

**+44**

171



46



slava_rumin

20 часов назад

Я ставлю датчик, иду на Авито и зарабатываю 2 млн в месяц на курьерах



Простой



10 мин



93K

Интервью



+34



62



79



interpres

21 час назад

Самохостинг и технологическая независимость



Простой



8 мин



7.5K

Обзор

Перевод



+29



79



7



virtual_explorer

1 час назад

Компании, заменившие людей на ИИ, стали осознавать свою ошибку



8 мин



1.4K



+23



14



15



Auterrix

2 часа назад

Оптимизация Redis для высоких нагрузок: полное руководство



Средний



15 мин



1K

Обзор



+18



22



0



letHabr



Облачный вызов

Соревнование разработчиков



Простой



4 мин



1.4K

Кейс

 +17 4 3**RakovskyAlexander**

14 часов назад

Про архитектуру приложений для тех кому мало Чистой архитектуры

**Сложный**

18 мин



4.5K

Обзор

 +15 82 13**iskndr_m**

21 час назад

Sigma + Trisigma = 4Sigma или как мы сделали эксперименты еще удобнее

**Средний**

4 мин



1.5K

Обзор

 +15 4 1

Исследование: как и где ML-специалисты используют ИИ. Расскажите о своем опыте

Опрос

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Облачный вызов

Соревнование разработчиков

как и зачем мы мигрировали на OVN

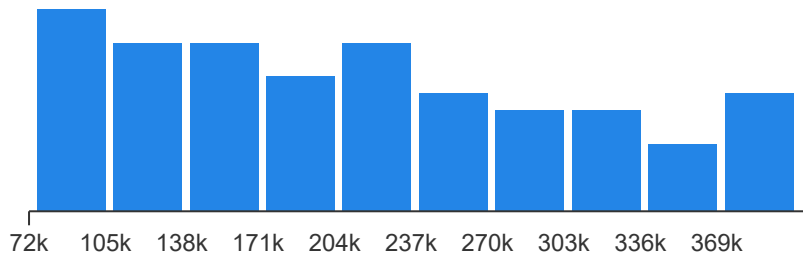
полный свайп: оцени, подходит ли тебе работа в IT-ритейле

Старт облачного соревнования. Призы на высоте!

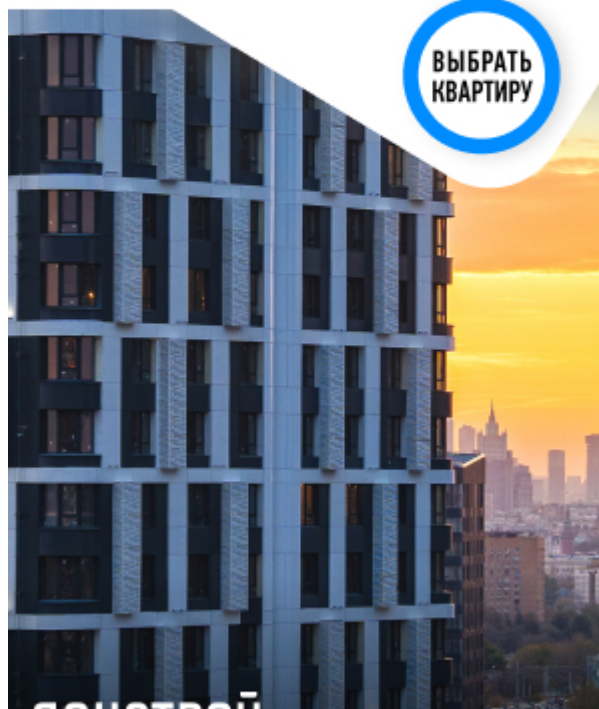
СРЕДНЯЯ ЗАРПЛАТА В IT

213 001 ₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 12 272 анкет, за 2-ое пол. 2025 года. Проверьте «в рынке» ли ваша зарплата или нет!

[Проверить свою зарплату](#)

РЕКЛАМА

СИМВОЛ**НОВЫЙ. ГОТОВЫЙ. В ЦЕНТРЕ МОСКВЫ**
ПАРК И ВСЯ ИНФРАСТРУКТУРА У ДОМА**Облачный вызов**

Соревнование разработчиков

ЧИТАЮТ СЕЙЧАС

Я ставлю датчик, иду на Авито и зарабатываю 2 млн в месяц на курьерах

93K 78

СМИ: восстановление IT-систем «Аэрофлота» может занять от нескольких недель до шести месяцев

15K 48

AGI математически невозможен, но хайп уже не остановить

43K 272

Кандидат сбежал в слезах. Про накрутку опыта

5.3K 74

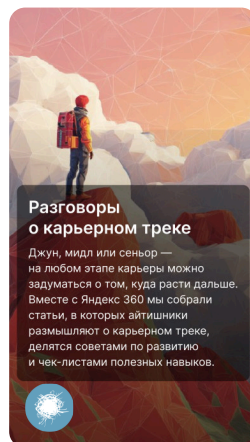
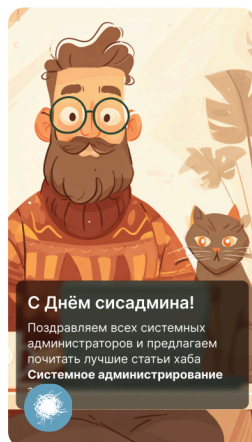
Экологичное расставание с Алисой: строим полностью локальный и приватный голосовой ассистент

16K 46

Исследование: как и где ML-специалисты используют ИИ. Расскажите о своем опыте

Опрос

ИСТОРИИ

**Облачный вызов**

Соревнование разработчиков

БЛИЖАЙШИЕ СОБЫТИЯ

Lerna

30 июля • 11:00 МСК • онлайн • Бесплатно

**Онлайн-конференция
«Не по шаблону»**

Как компании решают бизнес-задачи
через обучение: 4 живых кейса
+ дискуссия



30 июля

**Как компании решают
задачи бизнеса через
обучение — честные
кейсы от HR- и L&D-с**

Онлайн

Менеджмент

[Больше событий в календаре](#)

НЕБОЛЬНО

КОНФЕРЕНЦИЯ
ПРО КОНТЕНТ
В ИТ



31 июля

**«НЕБОЛЬНО» — онлайн-
конференция про контент
в ИТ**

Онлайн

Другое

[Больше событий в календаре](#)

Хабр

7 августа / 10:00

Мем-пати Хабр



7 августа

**Мем-пати: как
ИТ-бренд чере
контент и юм**

Москва

Маркетинг

[Больше событий в к](#)[Ваш аккаунт](#)[Разделы](#)[Информация](#)[Услуги](#)**Облачный вызов**

Соревнование разработчиков

[Компании](#)[Документы](#)

Авторы

Соглашение

Образовательные

Песочница

Конфиденциальность

программы

Стартапам



Настройка языка

Техническая поддержка

© 2006–2025, Habr



Облачный вызов

Соревнование разработчиков